

Introducción

Concepto de Integridade e elementos para garantila

Integridade: A información almacenada nunha BD debe ser:

- **Correcta:** O SXBD debe evitar que a BD conteña información errónea. EX: Non se permitirá un empregado con salario negativo.
- **Completa:** Toda a información requirida debe estar presente. Ex: Non se permitirá almacenar un empregado sen nome.

Para garantir a integridade, contamos con:

- **SXBD transaccional:** Permite que a BD pase de un estado consistente (garantido a integridade) a outro estado consistente cando remata a transacción. Durante a execución da transacción, de forma temporal, é posible que non se cumpla algunha regra.
- **Restricións de Integridade:** Son condicións duradeiras sobre os datos, especificadas de forma declarativa. O SXBD encárgase de verificalas de xeito automático.
- **BD Activa (Triggers ou disparadores):** Código programado que se executa no servior como resposta a un evento que se produce na BD. Utilízase normalmente cando o SXBD non é capaz de garantir determinada condición mediante restricións de integridade, e tamén de forma interna polo SXBD.

Integridade e fases de deseño da BD

As condicións de integridade deben incorporarse en **todas** as fases.

1. Modelado conceptual (ER)
 - Diagrama ER: Identificadores, cardinalidade e participación dos tipos de relación, ...
 - Documentación adicional (diccionario de datos): Condicións que non podemos pintar no ER.
2. Modelado lóxico (relaciona):
 - Claves primarias, claves foráneas, ...
3. Implementación física (SQL):
 - Restricións de Integridade
 - Triggers, se fosen necesarios.

Tipos de condicións de integridade (modelo físico)

En xeral as condicións para garantir a integridade impleméntanse en SQL, ben con restricións ou con triggers.

- Valores requeridos: Admisión ou non de nulos nun atributo.
- Valores únicos (non duplicados).
- Integridade de entidade e clave: A clave primaria, e as claves candidatas, non deben admitir nulos nin duplicados.
- Integridade referencial: Unha clave foránea conterá un valor existente na clave candidata, ou nulos.
- Validez dos datos (valores correctos):
 - Un atributo.
 - Unha fila (varios atributos).
 - Un dominio (se o SXBD os soporta).
- Outras condicións
 - Condicións que afectan a coleccións de filas e/ou táboas.
 - Regras de negocio ou condicións máis complexas establecidas no minimundo ou na organización.

Restricións nos modelos de datos

// Meter imagen de la diapositiva 7

Como se ve, no diagrama E-R aparecen restricións:

- Existen identificadores nos tipos de entidade (discriminadores se son débiles).
- Existen participacións e cardinalidades nos tipos de relación.

Pero existen certas restricións que non somos capaces de "pintar" no diagrama E-R. Por exemplo:

- O nome dos empregados é requerido.
- O nome do departamento é requerido e único, servirá como identificador.
- Restricións nos valores, como por exemplo que a data de fin dunha baixa debe ser posterior á data de inicio.

É importante recordar que o modelo conceptual non é só o diagrama E-R. Debe ir acompañado de máis documentación, como un diccionario de datos, que permite facer explícitas todas as restricións aplicables.

Restricións de integridade en SQL

Conceptos xerais

En xeral, en SQL podemos crear unha restrición:

- A nivel de columna (atributo): Ex: clave primaria dun só atributo que non admite nulos.
- A nivel de fila: Ademais de poder utilizalas para restricións que afectan a un só atributo, son necesarias cando hai varios implicados.

```
CONSTRAINT c_valid_date CHECK (ENDDATE > STARTDATE)
```

Ademais, podemos crear ou eliminar restricións:

- (Crear) No momento da creación da táboa, na sentencia `create table`.
- (Crear/Eliminar) Mediante a sentencia `alter table { add | drop} ...`

Toda restrición ten un nome, xa sexa indicado na sentencia SQL mediante `CONSTRAINT <nome>`, ou asignado polo sistema.

Por que é unha boa idea dar nomes ás restricións?

- Obtemos mensaxes de erro máis significativos cando hai unha violación da restrición.
`restricción de control (SYS_C0010357) violada vs. restricción de control (c_valid_date) violada`
- Permite unha xestión máis simple para eliminar ou habilitar/deshabilitar unha restrición. Tamén para poñela en modo aprazado/inmediato.
- Se creamos scripts SQL que, por exemplo, deshabilitan e habilitan restricións, é máis fácil facelos "repetibles" se usamos nomes.

Clave primaria (`PRIMARY KEY`)

- Unha clave primaria non admite nulos nin duplicados.
- En SQL especificase coa restrición `PRIMARY KEY`.
- Unha táboa só pode ter (como máximo) unha clave primaria.

Podemos crealas utilizando restricións sen nome:

```
-- "De columna"
CREATE TABLE DEPT(
    DEPTNO NUMERIC(2)
        PRIMARY KEY,
    DNAME ...
)

-- "De fila"
CREATE TABLE DEPT(
    DEPTNO NUMERIC (2),
    DNAME ...,
```

```
PRIMARY KEY (DEPTNO)
)
```

Ou, mellor, con nome:

```
CREATE TABLE DEPT(
    DEPTNO NUMERIC(2)
        CONSTRAINT pk_dept PRIMARY KEY,
    DNAME ...
)

CREATE TABLE DEPT(
    DEPTNO NUMERIC(2),
    DNAME ...,
    CONSTRAINT pk_dept PRIMARY KEY (DEPTNO)
)
```

Poderíamos eliminar restricións existentes, ou crealas novas, a posteriori:

```
ALTER TABLE DEPT DROP CONSTRAINT pk_dept;
ALTER TABLE LEAVE ADD CONSTRAINT pk_leave PRIMARY KEY(EMPNO, STARTDATE);
```

Note

Alguns xestores requiren que os atributos que forman a parte da clave inclúan a restrición `NOT NULL`. A maioría actívana de forma implícita.

Valores requiridos (`NOT NULL`)

Esta restrición é un tanto especial. Moitos xestores:

- En caso de violar a restrición, non inclúen o seu nome na mensaxe, só indican que se está intentando almacenar un nulo na columna, e que non se admite.
- Almacenan as restricións no catálogo como restricións do tipo `CHECK(campo is not null)`.
- Requiren modificar a columna asociada para crear/eliminar a restrición `NOT NULL`.

Unicidade (`UNIQUE`)

- Indica que un atributo, ou conxunto de atributos, non admite valores duplicados nos valores non nulos.
- O comportamento con respecto a duplicados nos nulos é uniforme nos distintos SXBD (Oracle, PostgreSQL e MySQL/MariaBD admiten duplicados nos nulos, SQL Server non).

- Unha restrición **UNIQUE** **non** implica que non se admitan nulos.
- Unha clave candidata formada por un conxunto de columnas implica a restrición **NOT NULL** de cada unha das súas columnas e a restrición **UNIQUE** de todo o conxunto de columnas.
- Unha restrición de clave primaria (**PRIMARY KEY**) implica unicidade.

```
ALTER TABLE EMP
  ADD EMAIL VARCHAR(50)
      CONSTRAINT u_email UNIQUE;

-- Clave candidata
CREATE TABLE DEPT(
  ...
  DNAME VARCHAR(20)
      CONSTRAINT nn_dname NOT NULL,
      CONSTRAINT u_dname UNIQUE,
  ...
)
```

Integridade referencial: clave foránea (FOREIGN KEY / REFERENCES)

- Unha restrición de clave foránea sobre un conxunto de atributos, referenciando unha clave candidata (normalmente, a clave primaria) dunha táboa, indica que os valores da clave foránea deben existir entre os valores da clave candidata, ou ser nulos.
- A clave foránea pode referenciar unha táboa diferente (no exemplo: **fk_dept**) ou a mesma táboa (no exemplo: **fk_mgr**).
- Definida a nivel de atributo: **REFERENCES táboa [(campos_cc)]**. Se omitimos os campos da clave candidata (**campos_cc**) referenciamos a clave primaria desa táboa.
- Definida a nivel de fila: **FOREIGN KEY (campos fk) REFERENCES táboa[(campos_cc)]**, onde **campos_fk** son os campos locais sobre os que se define a clave foránea.

```
CREATE TABLE EMP)
  ...
  DEPTNO NUMBER(2)      CONSTRAINT fk_dept REFERENCES DEPT(DEPTNO)
  MGR NUMERIC(4),
  -- Non indicamos o campo en EMP => será EMPNO, a clave primaria
  CONSTRAINT fk_mgr FOREIGN KEY (MGR) REFERENCES EMP
  ...
```

Accións referenciais

- Todos os demais tipos de restricións reaccionan só ante `INSERT` e `UPDATE` na propia táboa onde están definidas. Se se viola a restrición prodúcese un erro e abórtase a sentencia (ou transacción).
- Unha clave foránea permite controlar os seguintes cambios:
 1. Cambios na táboa que contén a clave foránea:
 - Inserción (`INSERT`) dunha nova fila.
 - Actualización (`UPDATE`) dun valor da clave foránea.

O comportamento da restrición é o habitual: Só se permite que o valor da clave foránea da fila insertada ou actualizada sexa nulo ou apareza entre os valores da clave candidata. En caso contrario prodúcese un erro.
 2. Cambios na táboa referenciada:
 - Borrado (`DELETE`) de filas da táboa referenciada.
 - Actualización (`UPDATE`) dun valor da clave candidata na táboa referenciada.

Neste caso, podemos especificar a acción que se vai a levar a cabo (comunmente denominada acción referencial), mediante:

 - `ON DELETE <acción>` para os borrados
 - `ON UPDATE <acción>` para as actualizacións.

Podemos establecer as seguintes accións referenciais:

- `NO ACTION`: Non se realiza a acción (borrado/actualización) e prodúcese un erro no caso de que os cambios violasen a integridade referencial. É a acción referencial predeterminada.
Caso especial: `RESTRICT`. Comproba a integridade de forma inmediata (`NO ACTION` esperaríase a que se executasen os triggers se os houbo, e pode aplazar a comprobación se a restrición está en modo aprazado).
- `CASCADE`: Traslada "en cascada" os cambios á táboa da clave foránea (borrando filas/actualizando valores).
- `SET NULL`: Actualiza a `NULL` os campos da clave foránea para as filas afectadas.
- `SET DEFAULT`: Actualiza os campos da clave foránea das filas afectadas ó valor predeterminado (`default`), se foi establecido.

```
-- Táboa LEAVE
CONSTRAINT fk_emp_leave FOREIGN KEY (EMPNO) REFERENCES EMP
  ON DELETE NO ACTION -- Predeterminada, pode omitirse
  ON UPDATE CASCADE;
```

Cando hai claves foráneas encadeadas ou incluso ciclos referenciais, debemos ter claro o comportamento, e as posibles implicacións.

- Claves foráneas encadeadas: No exemplo temos que `DEPT <- EMP <- LEAVE`

Que sucede se borramos un departamento (Considerando só `EMP.fk_dept` e `LEAVE.fk_emp_leave`)?

- Se ambas definen borrado en cascada
 - `(EMP) CONSTRAINT fk_dept REFERENCES DEPT ON DELETE CASCADE`
 - `(LEAVE) CONSTRAINT fk_emp_leave REFERENCES EMP ON DELETE CASCADE`Borraríase o departamento despois de borrar os empregados (e estes, despois de que borrasen as súas baixas).
- Se algunha define `NO ACTION`
 - `(EMP) CONSTRAINT fk_dept REFERENCES DEPT ON DELETE CASCADE`
 - `(LEAVE) CONSTRAINT fk_emp_leave REFERENCES EMP ON DELETE NO ACTION`Intentaría borrar o departamento. Para iso intenta borrar os empregados. Se algún ten unha baixa, a acción referencial `NO ACTION` de `fk_emp_leave` impide o borrado, co que non se borra nada (ningún departamento, empregado nin baixa) e prodúcese un erro.
- Ciclo referencial: Ocorre cando unha táboa se referencia a si mesma, xa sexa directa ou indirectamente a través de outras táboas. Exemplos:
 - `EMP` con `fk_mgr` que se referencia a si mesma.
 - `EMP` referencia a `DEPT (fk_dept)` e `DEPTNO` referencia a `EMP (fk_director)`

Posibles problemas:

- Borrados masivos recursivos: Se `fk_mgr` se define con acción referencial de borrado en cascada, borrar unha fila (a do xefe que está no máis alto da xerarquía) podería chegar a facer que se borrara toda a táboa `EMP`.
- Dificultade para introducir datos: Se `EMP.DEPTNO` e `DEPT.DIRNO` non admiten nulos, partindo dunha BD baleira, como introducimos datos?
 - Non se pode insertar un empregado se non existe ningún departamento para asociar (`not null + fk_dept`).
 - Non se pode insertar un departamento se non hai ningún empregado para ser director (`not null + fk_director`).

A cláusula `MATCH`

- Especifica a validez da clave foránea con respecto á candidata referenciada cando a clave foránea ten nulos.
- Hai tres tipos de `match`:
 - `MATCH SIMPLE`: (Predeterminado) Verifica a restrición se:
 - Algún dos campos é nulo, ou
 - Ningún campo é nulo e a clave foránea completa aparece na clave candidata.

- **MATCH PARTIAL** : Verifica a restrición se, para cada un dos seus campos:
 - É nulo, ou
 - Non é nulo e o valor dese campo aparece no campo correspondente da clave candidata.
- **MATCH FULL** : Verifica a restrición se:
 - Todos os campos son nulos, ou
 - Ningún campo é nulo e a clave foránea completa aparece na clave candidata.
- Como se pode comprobar, a cláusula **MATCH** só ten sentido en claves foráneas compostas de máis dun atributo, xa que se é un só os tres tipos coinciden.

```
-- Sobre unha suposta táboa de alegacións ás baixas
CONSTRAINT fk_leave_allegation FOREIGN KEY (EMPNO, STARTDATE) REFERENCES
LEAVE
  MATCH FULL
  ON DELETE NO ACTION ON UPDATE CASCADE;
```

Comprobación de valores (**CHECK**)

- As restricións **CHECK** permiten establecer os valores para:
 - Un atributo.
 - Unha fila (varios atributos, será unha restrición "de fila").
 - Un dominio, se o xestor os soporta.
 - Unha aserción (normalmente os xestores non as implementan).

```
-- Táboa EMP, restrición "de columna"
SAL NUMERIC(7, 2)
  CONSTRAINT c_valid_sal CHECK (SAL > 1080)
```

- Aínda que o predicado evaluado polo **CHECK** non ten restricións no SQL estándar, os xestores non implementan o uso de subconsultas.

```
CHECK (SAL <= (SELECT MAX(SAL) FROM EMP) );
-- Sobre EMP. Se fose implementable, que problema presenta?
CHECK (DEPTNO IN (SELECT DEPTNO FROM DEPT) );
-- Que pretende? Alternativas?
CHECK(UNIQUE (SELECT id FROM TAB1 UNION ALL SELECT id FROM TAB2));
```

- Algunha restrición, aínda que implementable, pode non ser a mellor idea.

```
CHECK (ESTADO IN ('APROBADO', 'DENEGADO'));

-- Lista de todas as provincias de España. Alternativas?
CHECK (provincia in ('A Coruña', 'Lugo', 'Ourense', 'Pontevedra',
'Asturias', 'Cantabria', ...))
```

Dominios

- Un dominio referencia un tipo de datos e (opcionalmente) unha restrición que especifica os valores válidos.
- Unha vez definido, poden crearse atributos que usan ese dominio.

```
CREATE DOMAIN salario NUMERIC(7,2)
    CONSTRAINT c_valid_sal_domain CHECK(VALUE > 1080);

CREATE TABLE EMP(
    ...
    SAL salario,
    ...
);
```

- Podemos engadir ou eliminar restricións asociadas a un dominio:
 - `ALTER DOMAIN <nome_dominio> ADD | DROP ...`
 - `ALTER DOMAIN salario DROP CONSTRAINT c_valid_sal_domain;`

Asercións

- Podemos pensar nunha aserción como una "restrición a nivel de BD".
- Créase con `CREATE ASSERTION <nome> CHECK (<condición>)`.
- A condición utilizará consultas `SELECT`.
- Reacciona ante `INSERT` e `UPDATE` nas táboas implicadas.
- Desafortunadamente, os SXBD non as implementan (ou teñen serios problemas).

Poden usarse triggers como alternativa.

```
CREATE ASSERTION as_similar_salaries
    CHECK( ( (SELECT MAX(SAL) FROM EMP) -
            (SELECT MIN(SAL) FROM EMP) ) < 100);

CREATE ASSERTION as_small_depts
    CHECK (NOT EXISTS
            (SELECT DEPTNO FROM EMP
```

```
GROUP BY DEPTNO
HAVING COUNT(*)>10));
```

Triggers

Un trigger, ou disparador, é tamén coñecido como unha regra ECA: Dispárase cando se produce un **Evento**, e se se verifica certa **Condición**, execútase unha **Acción**.

É unha programación baseada en eventos, non declarativo como as `constraints`.

Exemplo: Implementación da aserción `as_small_depts`:

SQL estándar

```
-- Necesitamos outro trigger para UPDATE
CREATE TRIGGER T_EMP_SMALL_DEPTS_AIS
AFTER INSERT ON EMP
FOR EACH STATEMENT
WHEN ( EXISTS (SELECT DEPTNO FROM EMP
                    GROUP BY DEPTNO
                    HAVING COUNT(*)>10))
SIGNAL SQLSTATE '99001',
        'Departamento demasiado grande';
```

Oracle

```
CREATE TRIGGER T_EMP_SMALL_DEPTS_AIUS
AFTER INSERT OR UPDATE ON EMP
DECLARE
    n NUMBER;
BEGIN
    SELECT COUNT(*) INTO N
    FROM (SELECT DEPTNO FROM EMP
          GROUP BY DEPTNO
          HAVING COUNT(*)>10);
    IF N > 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Departamento demasiado grande');
    END IF;
END;
```

Restricións aprazadas

- Unha restrición pode definirse como `DEFERRABLE` (aprazable, diferible), facendo que a comprobación se realice no momento momento de confirmar a transacción.

```
CONSTRAINT <nome> <definición>
    { NOT DEFERRABLE |
      DEFERRABLE INITIALLY { DEFERRED | IMMEDIATE}
    }
```

- O modo de comprobación predeterminado é **NOT DEFERRABLE**. Neste modo, (modo inmediato) a restrición compróbase no momento da execución da sentencia DML.
- Unha restrición aprazable especifica o modo no que comezaza cada transacción mediante **INITIALLY IMMEDIATE** (modo inmediato) ou **INITIALLY DEFERRED** (modo aprazado).
- Pode cambiarse o modo de comprobación (que ten efecto local, na transacción actual) a modo inmediato/aprazado mediante: ``SET CONSTRAINTS { ALL | restr1 [, restr2, ...] } { IMMEDIATE | DEFERRED}`
- Cando se executa unha sentencia DML que afecta a unha restrición:
 - En modo inmediato, compróbase. Se falla, prodúcese un erro e anúlase a sentencia.
 - En modo aprazado non se comproba. Cando se faga commit, compróbase (todas as fíals relevantes) e se falla, anúlase a transacción completa (faise **rollback**).

Exemplo:

```
create table test1(
    v numeric(3)
    constraint c_vpositive check (v > 0)
    deferrable initially deferred );

insert into test1 values(1); -- Non se comproba
insert into test1 values(0); -- Non se comproba
commit; -- Compróbase => fallo, rollback
select * from test1; -- Non hai filas

set constraints c_vpositive immediate; -- Modo inmediato
insert into test1 values(1); -- Comproba, Ok
insert into test1 values(0); -- Comproba, Fallo => anula insert
commit; -- Nada que comprobar, confirma
select * from test1; -- Está a fila con valor 1
```

Utilidade:

- No caso de que existan ciclos referenciais, con restricións de **not null**, entre dúas táboas. Ex: **EMP** e **DEPT**, poñendo **fk_dept** ou **fk_director** como aprazada.
- Cando se desexa realizar unha carga de datos que, unha vez rematada, verificará as regras de integridade, pero durante a carga é posible que non. Ex Cargar unha lista de empregados ordeada alfabeticamente (clave foránea **fk_mgr**).

Implementación en SXBD reais

Particularidades de Oracle

- Non implementa dominios
- Restricións: `PRIMATY KEY`, `NOT NULL`, `UNIQUE`, `FOREIGN KEY/REFERENCES`, `CHECK`.
- Poden consultarse as restricións propias no catálogo en `USER_CONSTRAINTS` e `USER_CONS_COLUMNS`. Tipo: `constraint_type`: P (primary), U (unique), R (references), C (check, inclúe as `not null`).
- Permite o modo aprazado seguindo o estándar.
- Activación e comprobación:
 - Unha restrición pode estar habilitada (`ENABLED`, predeterminado) ou non (`DISABLED`).
 - Unha restrición pode comprobar as filas existentes (`VALIDATE`, predeterminado) ou non (`NOVALIDATE`).
 - Pode especificarse no momento de creación ou modificarse con `ALTER TABLE`:
 - `ALTER TABLE <táboa> MODIFY CONSTRAINT <nome_restr> { ENABLED | DISABLED } { VALIDATE | NOVALIDATE };`
 - `ALTER TABLE <táboa> { ENABLE | DISABLE } CONSTRAINT <nome_restr>;`
- Claves foráneas:
 - Non permiten a cláusula `MATCH`. O comportamento é o de `MATCH SIMPLE`.
 - Non permite especificar acción referencial para `ON UPDATE` (comportamento: `NO ACTION`). (Poderían crearse triggers para implementar as demais accións referenciais).
 - Para o borrado, permite non especificar acción referencial (comportamento: `NO ACTION`) ou indicar `ON DELETE CASCADE` ou `ON DELETE SET NULL`.

Particularidades de PostgreSQL

- Implementa dominios
- Restricións: `PRIMATY KEY`, `NOT NULL`, `UNIQUE`, `FOREIGN KEY/REFERENCES`, `CHECK`, `EXCLUDE`.
- Poden consultarse as restricións no catálogo en `pg_catalog.pg_constraint` e varias vistas do esquema `information_schema`.
- Permite o modo aprazado con limitacións: non se admite para as restricións `CHECK` nin `NOT NULL`.

Note

Existe outra particularidade: Nun bloque transaccional, se falla unha sentencia a transacción está en "modo rollback" e nunca fará commit. Se hai unha restrición en modo inmediato e falla, falla a sentencia, pero se logo solicitamos un commit

realmente fará un rollback de toda a transacción. Podería evitarse mediante o uso de `savepoints`.

- Activación e comprobación:
 - Non permite activar/desactivar (directamente) unha restrición.
 - Sempre comproba as filas existentes cado se crea.
- Claves foráneas:
 - Cláusula `MATCH`: Están implementados `MATCH SIMPLE` (predeterminado) e `MATCH FULL`, pero non `MATCH PARTIAL`.
 - Accións referenciais: todas (`NO ACTION`, `RESTRICT`, `CASCADE`, `SET NULL` e `SET DEFAULT`) tanto para o borrado (`ON DELETE`) como para a actualización (`ON UPDATE`).